

Fundamentação Teórica

A construção de um motor de compreensão de linguagem natural (NLU) segue um encadeamento lógico: primeiro o texto precisa ser convertido em uma representação numérica; depois um algoritmo de Machine Learning aprende a associar essas representações a categorias, chamadas de intenções; em seguida, o modelo pode estimar o grau de confiança de suas previsões; por fim, uma camada de decisão determina se aquela previsão é suficientemente confiável para gerar uma resposta ou se o sistema deve acionar um mecanismo de segurança, o *fallback*.

1. Representação Textual e Vetorização com TF-IDF

Modelos tradicionais de Machine Learning não trabalham diretamente com palavras ou frases. Para o algoritmo, uma frase como:

"Quero consultar meu saldo"

é apenas uma sequência de caracteres.

Precisamos transformá-la em uma representação numérica que preserve, na medida do possível, a informação relevante do texto.

O **TF-IDF (Term Frequency – Inverse Document Frequency)** realiza justamente essa transformação.

Ele combina dois sinais:

- **TF (Term Frequency):** mede a frequência de um termo dentro de determinado documento.
- **IDF (Inverse Document Frequency):** reduz a importância de palavras que aparecem em muitos documentos e aumenta a importância de termos mais específicos.

A representação pode ser expressa por:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \log \left(\frac{|D|}{|\{d \in D: t \in d\}|} \right)$$

A ideia mais importante não é decorar a fórmula, mas compreender sua lógica:

Quanto mais específica for uma palavra para determinado contexto, maior tende a ser sua importância para o modelo.

Exemplo simplificado

Considere três mensagens:

- **D1:** "Quero consultar meu saldo"
- **D2:** "Quero fazer um pix"

- **D3:** "Quero consultar minha fatura"

A palavra "**quero**" aparece em todas as frases. Portanto, ela possui pouco poder para diferenciar as intenções.

Já:

- saldo
- pix
- fatura

aparecem em contextos muito mais específicos.

Consequentemente, essas palavras tendem a receber maior peso na representação TF-IDF.

Podemos imaginar o processo assim:

Mensagem

|



"Quero consultar meu saldo"

|



TF-IDF

|



Vetor numérico

|



Classificador

|



consultar_saldo

O ponto fundamental é que o classificador não "entende português" diretamente. Ele trabalha sobre relações matemáticas entre vetores.

Classificação de Intenções: Aprofundamento no KNN

Depois que as mensagens são transformadas em vetores, precisamos de um algoritmo capaz de relacionar esses vetores às intenções conhecidas.

Um dos algoritmos que podemos utilizar é o **K-Nearest Neighbors (KNN)**.

O conceito central do KNN é bastante intuitivo:

Uma nova mensagem tende a pertencer à mesma classe das mensagens de treinamento que estão mais próximas dela.

Diferentemente de modelos que aprendem uma função matemática explícita durante o treinamento, o KNN mantém os exemplos de treinamento e utiliza esses exemplos como referência quando recebe uma nova entrada.

Por isso, ele é conhecido como um algoritmo de **aprendizado baseado em instâncias** (*instance-based learning*) ou **lazy learning**.

Uma analogia simples

Imagine que você entrou em uma sala e encontrou dez pessoas.

Você não sabe quem é estudante de Ciência da Computação e quem é estudante de Biomedicina.

Entretanto, você percebe que está fisicamente muito próximo de cinco pessoas: Se a maioria dos seus vizinhos pertence à Biomedicina, uma estratégia simples seria classificá-lo como pertencente ao mesmo grupo.

O KNN faz algo semelhante, mas substitui a distância física pela **distância matemática entre vetores**.

O processo do KNN

Quando uma nova mensagem chega, o algoritmo realiza essencialmente quatro etapas:

1. Nova mensagem



2. Vetorização



3. Cálculo das distâncias



4. Seleção dos K vizinhos



5. Votação



6. Intenção prevista

Considere:

Mensagem nova:

"Quero acompanhar minha encomenda"

Depois da vetorização, temos:

$v_novo = [0.00, 0.72, 0.18, 0.00, \dots]$

O KNN compara esse vetor com os vetores existentes no conjunto de treinamento.

Suponha que os cinco vizinhos mais próximos sejam:

Vizinho 1 → rastreio

Vizinho 2 → rastreio

Vizinho 3 → troca

Vizinho 4 → rastreio

Vizinho 5 → rastreio

Temos:

RASTREIO → 4 votos

TROCA → 1 voto

Portanto:

Predição = RASTREIO

Por que a distância é tão importante?

O KNN não sabe o significado das palavras.

Ele sabe apenas calcular **distâncias entre representações numéricas**.

Em classificação de textos, uma métrica particularmente interessante é a **similaridade de cosseno**.

Ela mede o quanto dois vetores apontam para a mesma direção.

$$\text{Similaridade}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

O resultado normalmente varia entre:

1.0 → vetores muito semelhantes

0.0 → pouca ou nenhuma relação

Isso é particularmente interessante para textos porque duas frases podem possuir comprimentos diferentes e ainda assim apresentar uma estrutura semelhante.

Por exemplo:

"Quero consultar meu saldo"

"Gostaria de verificar o saldo"

As palavras não são exatamente iguais, mas os vetores podem apresentar características semelhantes dependendo da representação utilizada.

No código, quando utilizamos KNN com texto, podemos configurar explicitamente a métrica:

```
KNeighborsClassifier(  
    n_neighbors=3,  
    metric="cosine"  
)
```

Aqui existe um detalhe importante para os alunos:

O KNN não é obrigatoriamente baseado em distância de cosseno. A métrica depende da configuração do algoritmo. Em dados textuais vetorizados, a distância de cosseno é uma escolha bastante comum.

O problema do valor de K

A letra **K** representa quantos vizinhos serão considerados na decisão.

Por exemplo:

$K = 1$

O algoritmo considera apenas o vizinho mais próximo.

Com:

$K = 3$

considera os três vizinhos mais próximos.

Com:

$K = 5$

considera os cinco mais próximos.

Isso influencia diretamente o comportamento do modelo.

K muito pequeno

O modelo fica extremamente sensível aos exemplos individuais.

$$K = 1$$

Um único exemplo estranho pode influenciar completamente a decisão.

K maior

O modelo considera uma região maior do espaço.

$$K = 7$$

A decisão tende a ficar mais estável, mas pode perder características mais específicas dos grupos.

Portanto:

Escolher K é uma decisão de modelagem. Não existe um único valor universalmente correto.

O caso particular de K = 3

No nosso laboratório, quando utilizamos:

`n_neighbors=3`

temos uma consequência importante para a interpretação de `predict_proba()`.

Se dois dos três vizinhos pertencem à intenção rastreio e um pertence à intenção troca:

rastreio $\rightarrow 2/3$

troca $\rightarrow 1/3$

Portanto:

$$P(\text{rastreio}) = \frac{2}{3} \approx 66,7\%$$

e:

$$P(\text{troca}) = \frac{1}{3} \approx 33,3\%$$

Nesse cenário:

Predição: rastreio

Confiança: 66,7%

Com $K = 3$, as proporções de votos são discretas:

$$\left\{0, \frac{1}{3}, \frac{2}{3}, 1\right\}$$

ou:

0%

33,3%

66,7%

100%

Isso é importante porque mostra que a "confiança" do KNN não deve ser interpretada automaticamente como uma probabilidade perfeitamente calibrada.

Uma confiança de 66,7% significa, nesse caso:

2 dos 3 vizinhos considerados pertencem àquela classe.

Não significa necessariamente que exista uma probabilidade estatística real de exatamente 66,7% de o usuário possuir aquela intenção.

Esse detalhe será importante quando implementarmos o **fallback**.

Fallback e Limiar de Confiança (*Threshold*)

Um chatbot real precisa lidar não apenas com mensagens que conhece, mas também com mensagens que nunca foram previstas durante seu desenvolvimento.

Imagine que nosso chatbot bancário conheça apenas:

consultar_saldo

fazer_pix

consultar_fatura

bloquear_cartao

Agora o usuário escreve:

"Qual é a previsão do tempo para amanhã?"

Essa mensagem não pertence a nenhuma dessas intenções.

O problema é que o classificador não necessariamente responderá:

"Não conheço essa intenção."

Ele poderá ser obrigado a escolher uma das classes disponíveis.

Isso cria um problema importante de segurança e qualidade.

O problema do predict() sozinho

Imagine:

```
intencao = modelo.predict(nova_mensagem)
```

O modelo precisa retornar uma classe pertencente ao conjunto conhecido.

Se as classes disponíveis forem:

saldo

pix

fatura

bloqueio

ele escolherá uma delas.

Mesmo que a mensagem seja:

"Quem ganhou o campeonato ontem?"

O algoritmo poderá produzir:

fatura

Isso não significa que o modelo "entendeu" que a mensagem era sobre fatura.

Significa apenas que, entre as classes conhecidas, aquela foi a escolha considerada mais adequada segundo os critérios matemáticos do modelo.

Esse comportamento pode ser perigoso em um chatbot.

Imagine:

Usuário:

"Quero cancelar meu cartão."

Modelo:

consultar_fatura

Chatbot:

"Seu valor da fatura é R\$ 1.250..."

O sistema não apenas errou.

Ele **errou de maneira operacionalmente inadequada**.

Por isso precisamos de uma camada adicional de decisão.

A ideia do Fallback

O **fallback** funciona como uma barreira de segurança.

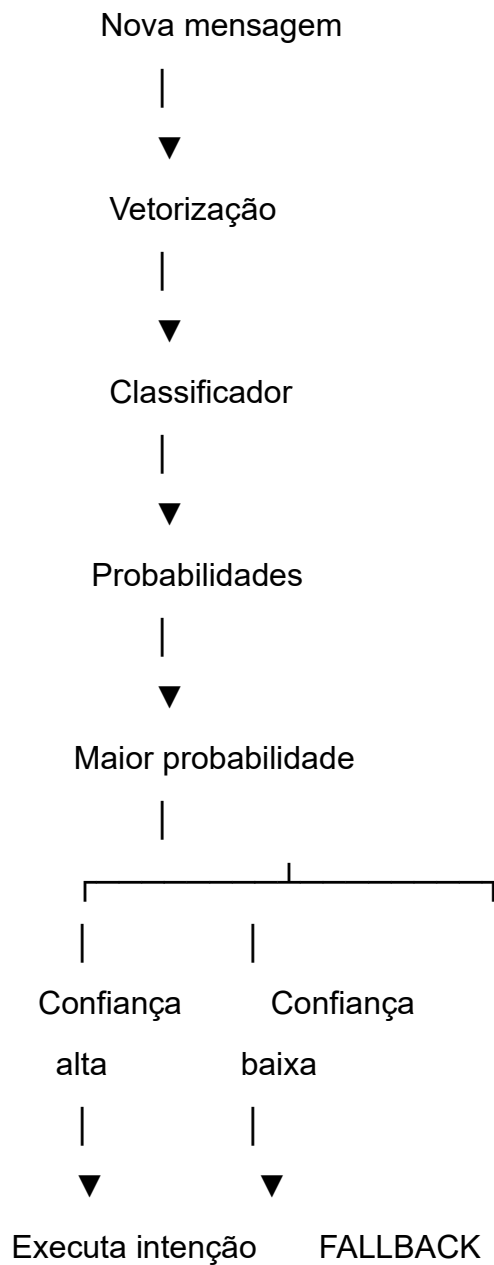
Em vez de perguntar apenas:

"Qual é a intenção mais provável?"

o sistema passa a perguntar:

"A intenção mais provável possui confiança suficiente para que eu execute uma ação?"

O fluxo passa a ser:



"Não entendi"

O fallback, portanto, não é exatamente uma característica do KNN.

Ele é uma **camada de decisão construída sobre o classificador**.

Essa distinção é importante:

KNN

↓

faz a classificação

Fallback

↓

decide se devemos confiar nessa classificação

Utilizando predict_proba()

Para implementar essa estratégia, precisamos conhecer não apenas a classe escolhida, mas também a distribuição de confiança entre as classes.

Por exemplo:

saldo → 0,00

pix → 0,67

fatura → 0,33

bloqueio → 0,00

O modelo seleciona:

pix

porque possui a maior pontuação:

$$P_{max} = \max(P)$$

Neste exemplo:

$$P_{max} = 0,67$$

Agora podemos estabelecer um limiar.

Threshold

Suponha que o sistema utilize:

THRESHOLD = 0,50

A regra será:

$$Decisao = \begin{cases} \text{Executar inten\c{c}ao}, & P_{max}' \geq Threshold \\ \text{Fallback}, & P_{max}' < Threshold \end{cases}$$

Na prática:

if confianca >= 0.50:

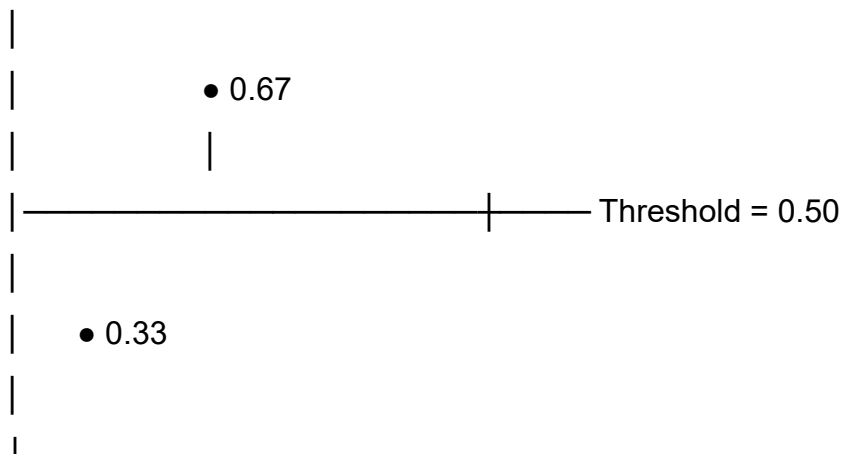
 executar_intencao()

else:

 executar_fallback()

Podemos visualizar:

Confiança



No primeiro caso:

0.67 >= 0.50

Resultado:

Executar intenção

No segundo:

0.33 < 0.50

Resultado:

Fallback

O threshold não deve ser escolhido arbitrariamente. Ele representa um compromisso entre dois comportamentos indesejáveis:

Threshold baixo



menos fallbacks



mais respostas erradas

Enquanto:

Threshold alto



mais fallbacks



menos respostas erradas

Podemos imaginar:

Threshold Comportamento

0,30	Bot responde mais, mas assume mais riscos
0,50	Equilíbrio inicial
0,70	Bot exige maior confiança
0,90	Bot torna-se muito conservador

No KNN com $K=3$, entretanto, existe uma particularidade. Como as probabilidades são baseadas na quantidade de vizinhos, podemos ter:

$K = 3$

0%

33,3%

66,7%

100%

Assim, um threshold de 0.50 produz uma consequência interessante:

33,3% → FALLBACK

66,7% → ACEITA

Não existe uma confiança intermediária entre esses dois valores.

Esse é um dos motivos pelos quais devemos compreender o comportamento do algoritmo antes de simplesmente escolher um threshold.

Exemplo completo

Considere o seguinte conjunto de intenções:

saldo

pix

fatura

O usuário envia:

"Quero consultar meu dinheiro"

O sistema executa:

Texto

↓

TF-IDF

↓

Vetor numérico

↓

KNN

↓

predict_proba()

Resultado:

saldo → 66,7%

pix → 33,3%

fatura → 0%

A maior confiança é:

66,7%

Com:

threshold = 50%

temos:

66,7% $\geq$ 50%

Portanto:

INTENÇÃO = saldo

AÇÃO = executar consulta de saldo

Agora imagine:

"Qual é a receita de bolo de chocolate?"

O modelo pode produzir algo como:

saldo $\rightarrow$ 33,3%

pix $\rightarrow$ 33,3%

fatura $\rightarrow$ 33,3%

Nesse caso:

$P_{\max} = 33,3\%$

Como:

33,3% $<$ 50%

o sistema não deve escolher arbitrariamente uma intenção.

Ele deve acionar:

FALLBACK

"Desculpe, não consegui entender sua solicitação.

Pode reformular a pergunta?"